

Bosy: Bounded Synthesis Framework

Authors : Peter Faymonville, Bernd Finkbeiner, and Leander Tentrup
Venue : SYNTCOMP tool competition at CAV 2016

Team Members: Devansh, Tarun, Meet

Under the guidance of:
Professor **Dr. Ashish Mishra**

- **Problem:** Lack of existing efficient tool standard for bounded synthesis
- **Solution:** Bosy – A framework for Bounded synthesis

Architecture

- **Input:** LTL specification in JSON format
- **Preprocessing:** Tool searches for a system *strategy* with ϕ and an environment *counter-strategy* with $\neg\phi$ in parallel.
- **Translation:** After parsing, LTL formula is translated into an equivalent universal co-Büchi automaton using an external tool.
- **Encoding:** Responsible for creating the constraint system based on the selected encoding.
- **Solver:** A tool that will solve the produced constraint system.
- **Postprocessing:** Extracts the implementation from a satisfiable result.

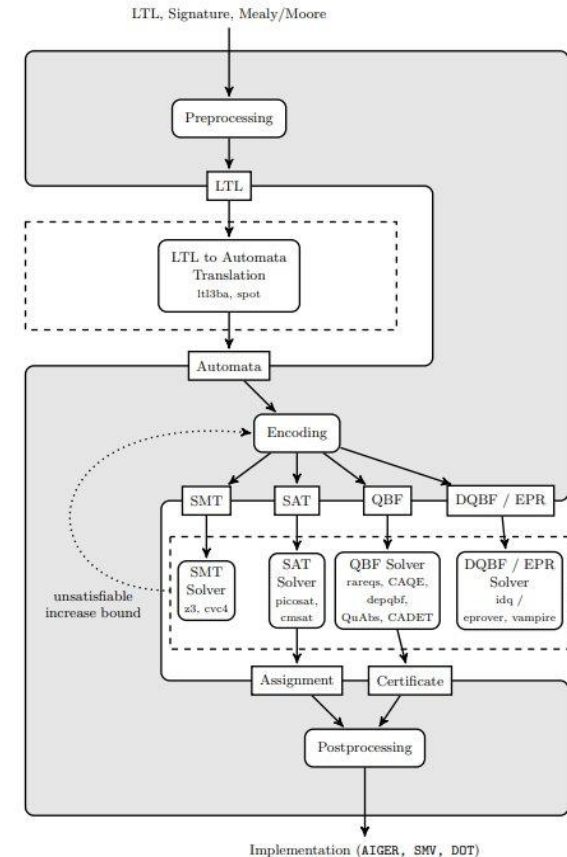


Image taken from the BoSy paper.

1. Efficient Bound Search

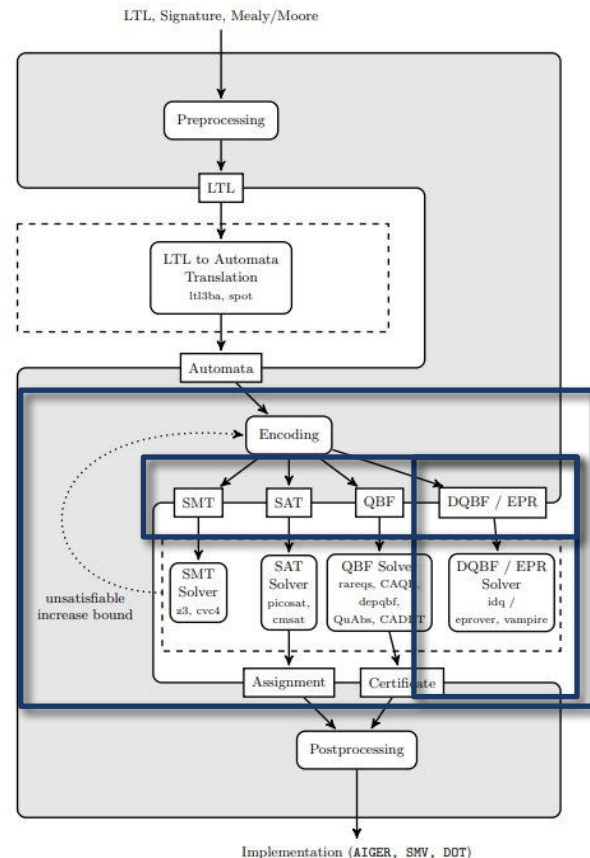
- A new schedule to increment n : exponential growth and binary search.
- Example: $n = 1, 2, 4, 8, 16, \dots$
- If solution is not found at 8 but found at 16, we commence binary search for the minimal satisfying n between 8 and 16.

2. Understanding DQBF landscape

- Test out modern DQBF solvers and compare them to the paper's results.
- Understand how these solvers have since developed and comment on these tools.

3. Can LLMs solve?

- How strong are LLMs at solving encodings?
- Can they be included at any point in the pipeline?



Experiment 1: Bound Search Schedule

Initial Hypothesis:

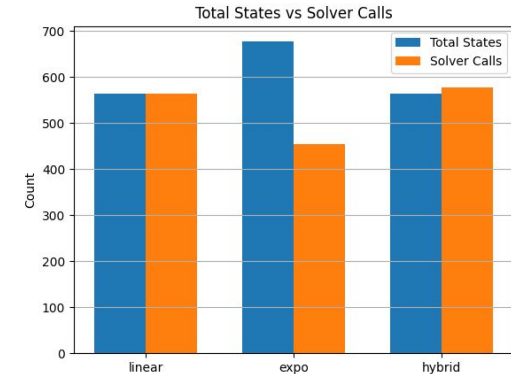
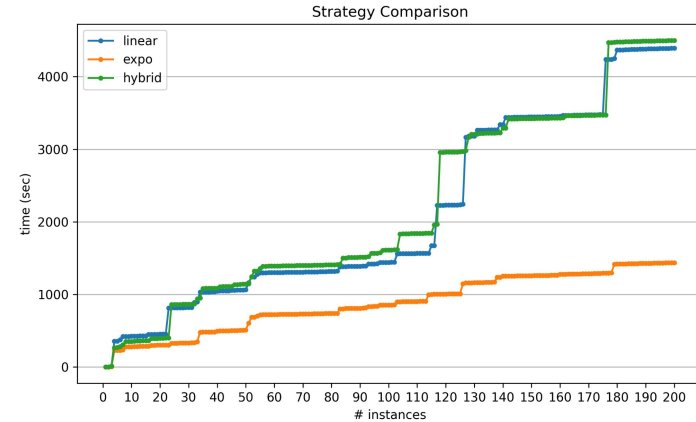
- Linear schedule is too slow, exponential won't get the minimal solution.
- A Hybrid approach with Exponential search + Binary search is better

Experiment:

- Run each strategy for 200 instances of the SYNTCOMP 2019 dataset with the default solver (game-solving)
- Tracked time taken per instance in the SYNTCOMP 2019 dataset.

What we found:

- **Exponential** search achieves the **lowest runtime and fewest solver calls**, but **produces larger solutions** (more states).
- Linear and Hybrid strategies show similar runtime performance.
- **Hybrid** search requires slightly **more solver calls** than **Linear search**, but produces the same minimal-state solutions.
- Since most benchmarks have maximum states ≤ 8 , **Hybrid search does not show significant benefits** on these workloads.
- Hybrid search is expected to be more effective for benchmarks with larger state spaces (> 16 states).



Experiment 2: DQBF Solver exploration

Initial Hypothesis:

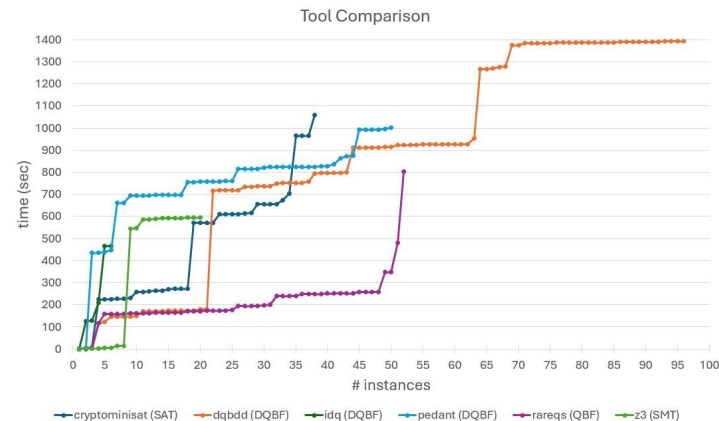
- The paper states that DQBF solvers were immature at the time of the paper's release.
- We hypothesized that recent DQBF solvers could possibly perform better given the paper is quite old.

Experiment:

- Run each solver for 3 hours on the SYNTCOMP 2019 dataset with a timeout of 10 minutes.
- We took two recent solvers: [Pedant](#) and [DQBDD](#), (from 2022 and on).
- We calculated the total time taken from LTL input to realizability checking.

What we found:

- DQBDD and Pedant **both** perform better than iDQ, a DQBF solver used in BoSy.
- Though, it seems as though DQBDD's results are not as reliable, as there seem to be some discrepancies in its results.
- Pedant, a modern DQBF solver, considerably improved upon the performance of iDQ (from 2017), competing with SAT solvers.
- Pedant even comes close to RAREQS in terms of total instances solved, but took more time to solve them.
- Conclusion: DQBF solvers have improved considerably, though QBF solvers still remain more efficient



Experiment 3: Can LLMs solve?

Initial Hypothesis:

- LLMs can parse SMTLIB2 formulas and produce satisfying assignments, acting as a drop-in replacement for traditional SMT solvers.
- Larger/reasoning-capable models (gpt-4o, o4-mini) would handle the quantified synthesis constraints better than smaller models.

Experiment:

- Integrated an LLM backend into BoSy that sends the full SMTLIB2 encoding to the OpenAI API, with the model asked to return SAT/UNSAT and a JSON map of variable assignments.
- Tested on simple_arbiter (3-input, 3-output Mealy arbiter, correct solution at bound 4) under different prompt framings with gpt-4o and o4-mini.

What we found:

- LLMs respond to prompt incentives rather than formula content — prompting gpt-4o to **"find a satisfying assignment"** produced a spurious SAT at bound 1 (where no solution exists).
- reframing with **"wrong SAT is worse than honest UNSAT"** caused it to return UNSAT at every bound, including bounds well above the correct solution bound of 4.
- Both models hallucinated SAT at bounds where no solution exists, always returning the cheapest valid-looking output: all-false grants with an input-blind transition — satisfying mutual exclusion trivially while ignoring liveness constraints entirely.



Thank You

